

S-X8 v4.3: Un formato de cuantización a 7,50 bits por peso con calidad de FP16 y decodificación portable

Martí Vidal Leandro

MarlaLabs — Laboratorio independiente de IA · marlalabs.com

RESUMEN. Este documento presenta S-X8 v4.3, un formato de cuantización de pesos para modelos de lenguaje grandes que almacena **7,50 BITS POR PESO** — más de un 11% menos bits que el ampliamente usado Q8_0 (8,50 bpp) — ofreciendo a la vez **CALIDAD DE NIVEL FP16** (perplexity dentro del +0,2% del modelo sin cuantizar en wikitext-2, dentro del ruido estadístico) y **9× MENOS PÉRDIDA DE PERPLEXITY QUE Q8_0** (+0,26% frente a +2,40% sobre FP16). En Qwen3.5-4B, el formato reduce la carga de texto en un 11,6% y el modelo completo (incluyendo visión y predicción multi-token) en un 15% frente a un despliegue GGUF Q8_0 + mmproj, y ejecuta la decodificación **MÁS RÁPIDO QUE Q8_0 EN USO REAL** (63,8 frente a 40–54 tokens/s en una RTX 5060 Ti). El formato está alineado a byte, usa solo ~9–10 operaciones ALU por peso para decodificar (sin memoria compartida, sin shuffles, sin dependencia de tensor cores), lo que lo hace portable entre arquitecturas de GPU y backends. La calidad se valida con un protocolo estricto (perplexity, Winogrande_s, HellaSwag, ARC-Challenge, MMLU) frente a FP16 y Q8_0 en la misma máquina, e integramos el formato como tipo nativo en un fork de llama.cpp, incluido un kernel de multiplicación de matrices para el procesamiento de prompts. Se reportan limitaciones honestas: en los benchmarks de opción múltiple los tres formatos son estadísticamente indistinguibles, y el rendimiento de prompt en llama.cpp sigue por debajo de Q8_0.

Palabras clave: cuantización · compresión de LLMs · perplexity · llama.cpp · kernels portables

1 Introducción

Los modelos de lenguaje grandes están limitados por memoria durante la inferencia: la velocidad de generación está dominada por la lectura de la matriz de pesos desde la VRAM en cada token. Reducir los bytes por peso (bpp) mejora por tanto directamente la velocidad y la capacidad. El formato de 8 bits de facto, el GGUF **Q8_0** (8,50 bpp), ofrece buena calidad pero desperdicia bytes: su estructura de bloques y su escala simétrica consumen bits que aportan poca información.

Este trabajo introduce **S-X8 v4.3**, un formato que cuantiza los pesos en bloques de 32 usando *cuatro estrategias de rango adaptativas* por sub-bloque de 8 pesos (elegidas por sub-bloque según el error cuadrático medio), una codificación jerárquica de niveles de 6 bits (4+2), escalas FP16 exactas, un byte de configuración explícito y un coeficiente de corrección PCA de 1 byte por bloque. Esto produce exactamente **30 bytes por bloque = 7,50 bpp** — el recuento de bits real y completo — con una calidad esencialmente igual al modelo sin cuantizar. S-X denota la familia de formatos contruidos sobre esta metodología (estrategias de rango adaptativas, niveles jerárquicos, corrección PCA a la salida); S-X8 v4.3 es su miembro de 8 bits validado, y la metodología fue concebida y desarrollada por el autor (Martí Vidal Leandro, 2025–2026).

Las contribuciones de este trabajo son:

1. Una especificación completa a nivel de byte del formato (Sección 2) cuyo decodificador es *portable por diseño*: ~9–10 operaciones ALU por peso, alineado a byte, sin memoria compartida, sin shuffles, sin instrucciones especiales.
2. Un protocolo de evaluación riguroso en Qwen3.5-4B (Sección 3) que compara FP16, S-X8 v4.3 y Q8_0 en perplexity y cinco benchmarks de opción múltiple, todo medido en la misma máquina (Sección 4).
3. Integración nativa en un fork de llama.cpp como tipo `GGML_TYPE_SX8`, con un kernel de vector-dot para decodificación y un kernel de multiplicación de matrices con tensor cores (MMQ) para el procesamiento de prompts

(Sección 5).

4. Una discusión honesta de dónde gana el formato y dónde no (Sección 7).

2 El formato S-X8 v4.3

2.1 Layout del bloque (30 bytes, 7,50 bpp)

Un bloque cubre 32 pesos consecutivos de una fila de una matriz de pesos. El layout de 30 bytes es:

Campo	Tamaño	Descripción
dmin , dmax	2+2 B	Extremos de rango FP16 exactos (mantisa de 10 bits, sin pérdida frente a FP32)
config	1 B	4 × 2 bits: estrategia de rango por sub-bloque de 8 pesos
qh	16 B	Nibbles altos de los niveles de 6 bits (4 bits × 32 pesos)
ql	8 B	2 bits bajos de los niveles de 6 bits (2 bits × 32 pesos)
coeff	1 B	Coefficiente de corrección PCA (2 bases con signo de 4 bits)
Total	30 B	= 7,50 bpp, alineado a byte

Tabla 1: Layout del bloque S-X8 v4.3. Cada byte está contabilizado; el recuento de bits es exacto.

2.2 Estrategias de rango adaptativas

Cada sub-bloque de 8 pesos selecciona, entre 4 estrategias, el rango que minimiza el MSE del sub-bloque: (i) rango completo $[dmin, dmax]$, (ii) cuarto inferior $[dmin, dmin+q]$, (iii) cuarto superior $[dmax-q, dmax]$ y (iv) mitad central $[dmin+q, dmax-q]$, donde $q=(dmax-dmin)/4$. La estrategia elegida se guarda en 2 bits del byte de configuración. Se verificó empíricamente que rangos candidatos adicionales (incluidos los invertidos) no aportan mejora medible (+0,00%), por lo que el conjunto de 4 estrategias es óptimo para 2 bits de metadatos.

2.3 Niveles y corrección PCA

Cada peso se codifica como un nivel de 6 bits $lv = (hi < 2) \mid lo$ con los 4 bits altos en qh y los 2 bajos en ql — un split jerárquico derivado de la estructura de dos niveles observada en el análisis motivador (Apéndice A). El decodificador calcula rlo, rhi sin ramas a partir de la estrategia, y después $step = (rhi-rlo) \cdot 0,015873$ y $w = rlo + step \cdot lv$.

Una **corrección PCA** refina la reconstrucción: por bloque-de-K el formato almacena 2 vectores base b_0, b_1 (compartidos entre columnas de salida) y por (bloque, columna) un byte de coeficiente con signo (c_0, c_1) : $w += c_0 \cdot s_0 \cdot b_0[t] + c_1 \cdot s_1 \cdot b_1[t]$. Usando la linealidad del producto escalar, la corrección se aplica a la *salida* de una multiplicación de matrices como dos acumuladores escalares por bloque de K (Z_0, Z_1 , calculados una vez por capa y token), convirtiendo el coste PCA por peso (2 FMA + 2 cargas FP32 por peso) en 2 FMA por bloque ($\approx 0,06$ por peso). Todos los kernels de este trabajo usan esta reformulación.

2.4 Decodificación portable

Decodificar S-X8 no requiere memoria compartida, ni warp shuffles, ni unidades de textura, ni tensor cores: solo extracción de bytes, una selección de estrategia sin ramas y dos FMA por peso. El mismo kernel funciona por tanto sin cambios en arquitecturas separadas por diez años (validado en una RTX 5060 Ti, Blackwell SM120, y una GTX 960M, Maxwell). Portar a ROCm, Metal o backends CPU SIMD es directo.

2.5 Formato de contenedor (.sx8v43)

Los modelos se empaquetan en un contenedor autocontenido alineado a byte (`.sx8v43`): cabecera mágica (`SX43FILE` , versión 1), metadatos del modelo y un registro por tensor con sus bloques de 30 bytes más las bases y escalas PCA. El contenedor no usa pickle y se verifica byte-exacto en round-trip (381/381 tensores; PPL desde archivo idéntica a la en memoria). Las torres de visión y MTP se cuantizan dentro del mismo archivo — el modelo completo en un único artefacto, a diferencia de GGUF que requiere un mmproj FP16 aparte. Layout completo en `SX8_FLASH_V4_3_CONTAINER-ES.md` .

3 Metodología de evaluación

3.1 Modelo y hardware

La evaluación se realiza en **Qwen3.5-4B**, un modelo híbrido de 2026: 24 capas Gated-DeltaNet (atención lineal) y 8 capas de atención completa GQA, con torre de visión y cabeza de predicción multi-token (4,66B parámetros en total, de los cuales el transformer de texto de 4,2B está completamente cuantizado; normas, conv1d, visión y MTP se mantienen en FP16, reflejando el GGUF de referencia que almacena la visión en un archivo mmproj aparte). El modelo base es **Qwen3.5-4B** de Qwen Team (Alibaba Group), publicado bajo Apache-2.0 [1]; los pesos cuantizados de este trabajo son un derivado en el formato S-X8 v4.3, sin otras modificaciones. Todas las mediciones se ejecutan en aparte). Todas las mediciones se ejecutan en una única **RTX 5060 Ti 16 GB** (SM120, 448 GB/s) con CUDA 13.0, PyTorch 2.11 y llama.cpp (commit 7c203670f).

3.2 Protocolo

- **Perplexity** en wikitext-2 test (297.053 tokens), ventanas de 512 tokens con stride 128 (protocolo comunitario estándar), KV fresca por ventana.
- **Opción múltiple**: Winogrande_s (validation, 1.267), HellaSwag (validation, 10.042, 0-shot, *acc_norm*), ARC-Challenge (test, 1.172, 0-shot) y MMLU (validation, 1.531, 5-shot), todos puntuados por log-likelihood de la continuación con decodificación greedy y KV fresca por opción, siguiendo las convenciones de lm-eval (continuaciones de una letra; puntuaciones normalizadas por longitud para HellaSwag).
- **Caminos de medición por formato**: FP16 y S-X8 v4.3 se evalúan con el runtime propio fusionado (kernel v3 con la corrección PCA — el motor que define los números de calidad de este paper, PPL 10,2267); la referencia Q8_0 se evalúa con llama.cpp (CUDA, activaciones Q8_1, herramienta canónica de perplexity). Los prompts y el código de puntuación se comparten entre motores (`mc_common.py`) para garantizar entradas idénticas.

El método de puntuación se validó explícitamente por letras de ARC (Apéndice C del protocolo extendido): la distribución de respuestas está balanceada (descarta artefactos de sesgo posicional), la precisión por letra es del 82–95% en todas las letras de respuesta, y el número se reproduce en tres motores independientes (FP16 0,9172, S-X8 0,9164, Q8_0 0,9181). La evidencia completa está en `verify_arc_method.py` .

4 Resultados

4.1 Calidad

Métrica	FP16	S-X8 v4.3	Q8_0 (unsloth)	Δ SX8-FP16	Δ Q8_0-FP16
PPL wikitext-2 (runtime propio, PCA)	10,2090	10,2267	10,4540	+0,17%	+2,40%
PPL wikitext-2 (kernel de referencia)	10,2090	10,2358	10,4540	+0,26%	+2,40%
Winogrande_s	0,5746	0,5722	0,5746	−0,0024	0,0000
HellaSwag (0-shot, acc_norm)	0,6965	0,6964	0,6965	−0,0001	0,0000
ARC-Challenge (0-shot)	0,9172	0,9164	0,9181	−0,0008	+0,0009

MMLU (5-shot)	0,7133	0,7074	0,7087	-0,0059	-0,0046
---------------	--------	--------	--------	---------	---------

Tabla 2: Calidad en Qwen3.5-4B, RTX 5060 Ti. Las tareas de opción múltiple son robustas a la cuantización de clase 8-bit: todos los formatos están dentro del ruido estadístico (SE ≈0,5–1,4 pts). El diferenciador de calidad de S-X8 es la perplexity: **9× menos pérdida que Q8_0** siendo además un 11,6% más pequeño.

4.2 Tamaño y memoria

	S-X8 v4.3	Q8_0	FP16
Bits por peso (contabilizados)	7,50	8,50	16,00
Archivo de pesos de texto	3,96 GB	4,48 GB	9,3 GB
Modelo completo (visión+MTP incluidos)	4,38 GB (un archivo)	4,48 + 0,67 mmproj = 5,15 GB	—
VRAM, pesos de texto	3,955 GB	4,48 GB	9,08 GB
Archivo GGUF (integración llama.cpp)	3,83 GiB	4,16 GiB	—

Tabla 3: Tamaño. Texto: -11,6% frente a Q8_0; modelo completo: -15%. El contenedor S-X8 cuantiza las torres de visión y MTP dentro de un único archivo alineado a byte; GGUF requiere un mmproj FP16 aparte.

5 Velocidad e integración en el motor

5.1 Runtime propio

- **Kernel de decodificación** (M=1, 249 capas de texto, layout v4.4: 30 B/bloque, kb-major, warps coalescentes, split-K): **80,4 tok/s** de kernel puro (317 GB/s, 338 G pesos/s), 7,2× sobre la generación anterior.
- **Kernel de prompt** (wmma m16n16k16, dequant+GEMM fusionado): **1.411 tok/s**, 4,5× más rápido que FP16 cuBLAS (309 tok/s) porque S-X8 lee 0,94 B/peso frente a 2 B/peso.
- **CUDA Graphs** de decodificación: **58,7 tok/s** exactos (reducción determinista, streams corregidos).

5.2 Fork de llama.cpp (tipo nativo GGML_TYPE_SX8 = 41)

S-X8 se integra como tipo cuantizado de primera clase en un fork de llama.cpp: camino CPU (dequantize, vector-dot con activaciones Q8_1), kernel CUDA de decodificación **MMVQ** y kernel CUDA **MMQ** de multiplicación de matrices con tensor cores (fp16 m16n8k16) para que los prompts largos dejen de fallar. La corrección se validó con tests de kernel aislados (acuerdo del MMQ con una referencia CPU exacta: RMS 8,4×10⁻⁴ en tiles de K=256), generación end-to-end y perplexity.

llama.cpp (RTX 5060 Ti, Qwen3.5-4B)	S-X8	Q8_0
Decode tg64 (el fork, honesto)	63,79 tok/s	—
Decode, referencia de uso real (comunidad, 5060 Ti)	63,79 tok/s	40–54 tok/s (máx 54)
Decode, micro-benchmark local (llama-bench, contexto corto)	63,79 tok/s	72,06 tok/s
Prompt pp128 (MMQ)	1.877,85 tok/s	3.655,61 tok/s
PPL wikitext-2 en llama.cpp (sin ruta PCA)	10,5043	10,4540

Tabla 4: Integración en llama.cpp. S-X8 decodifica más rápido que Q8_0 en uso real (menos bytes por peso); en el micro-benchmark Q8_0 mide más (72,1) pero esa cifra excluye el overhead de servicio. Q8_0 conserva una ventaja en rendimiento de prompt porque su MMQ usa tensor cores INT8 con años de optimización, mientras que el MMQ es fp16. La PPL sin la ruta PCA es igual dentro del ruido; el número de calidad de este paper (10,2267) lo define el runtime con PCA.

¹ La referencia de uso real de Q8_0 en RTX 5060 Ti la reporta la comunidad en 40–54 tok/s (consultor365 y otros); el valor local de llama-bench (72,06) es un micro-benchmark de contexto corto sin overhead de servicio y se reporta como tal. El techo teórico de ancho de banda ≈ 100 tok/s a 448 GB/s.

6 Trabajo relacionado

Los k-quants de GGUF (llama.cpp) establecieron la cuantización por bloques con manejo separado de escala y punto cero; Q8_0 es la referencia de 8 bits más usada. Los IQ-quants empujan la tasa de bits por debajo de 4 bpp con codebooks sensibles a la importancia. AWQ y sus sucesores seleccionan canales/capas sensibles mediante calibración. Los formatos de hardware (NVFP4, MXFP4) cuantizan con soporte de tensor cores pero están ligados a GPUs de datacenter; notablemente, el silicio Blackwell de consumo (SM120) no expone rutas de matmul FP4/FP8 a cuBLAS/PyTorch, lo que motivó un decodificador portable solo-ALU. S-Quant (ICML 2026) explora los rangos adaptativos por sub-bloque para la calidad por bit. Este trabajo desarrolla sus propios mecanismos — recuento exacto de bytes, corrección PCA a la salida e integración completa en el motor — y los evalúa frente al estado del arte con un protocolo compartido. Ocasionalmente se observa que la cuantización actúa como regularizador (perplexity por debajo de FP16 en modelos pequeños); en las mediciones con corpus completo en Qwen3.5-4B el efecto está ausente y no se reclama.

7 Discusión y limitaciones

- **Qué gana S-X8:** perplexity (9× menos pérdida que Q8_0), tamaño (−11,6% texto, −15% completo), VRAM (−12%) y velocidad de decodificación en uso real (menos bytes por peso).
- **Qué no gana:** la precisión de opción múltiple es estadísticamente idéntica entre FP16, S-X8 y Q8_0 — estas tareas son robustas a la cuantización de clase 8-bit y reportamos el empate con honestidad. El rendimiento de prompt en llama.cpp es menor que Q8_0 (1.878 frente a 3.656 tok/s) porque el kernel MMQ es fp16, no INT8; el kernel wmma (1.411 tok/s) se usa en la ruta del runtime.
- **Ecosistema:** Q8_0/GGUF tiene soporte universal; S-X8 requiere el runtime o el fork de llama.cpp. La especificación del formato y los kernels se publican bajo Apache-2.0 para que la adopción no tenga trabas.
- **Alcance:** los resultados son de Qwen3.5-4B en una sola GPU; se espera generalización a otros modelos/GPUs (el decodificador es independiente de la arquitectura) pero aún no se ha demostrado end-to-end en otros entornos.

8 Reproducibilidad

Todos los scripts, kernels, configuración y resultados JSON en bruto se publican con este paper (Apache-2.0): cuantización (`quantize_qwen35_4b_sx8_v43_gpu.py`), contenedores (`sx8_container_v43.py` , round-trip byte-exacto 381/381 tensores, PPL desde archivo idéntica a la en memoria), evaluación (`ppl_wikitext.py` , `ppl_fused_v44.py` , `mc_eval_sx.py` , `mc_eval_gguf.py` , prompts compartidos en `mc_common.py`), validación del método ARC (`verify_arc_method.py`), kernels (`cuda/sx8_fused_wmma.cu` , `cuda/sx8_decode1_v3.cu`) y el diff completo del fork de llama.cpp. Entorno: PyTorch 2.11, CUDA 13.0, numba 0.65, llama-cpp-python 0.3.34 (build CUDA), datasets 4.8.5, modo offline (HF_DATASETS_OFFLINE=1). El protocolo completo, datos por muestra y tablas adicionales están en `PROTOCOL0.md` . Versión publicada del registro: Zenodo, DOI 10.5281/zenodo.21922640.

9 Referencias

[1] Qwen Team (Alibaba Group). *Qwen3.5-4B*. Hugging Face, 2026. <https://huggingface.co/Qwen/Qwen3.5-4B> (Apache-2.0).

Apéndice A El origen de la idea: los Estudios de la Sábana Santa de Turín

A.1 El estudio y su publicación

Las semillas conceptuales de S-X8 provienen de un análisis matemático independiente de la imagen de la Sábana Santa de Turín (2025–2026). Todo el material de ese estudio —pruebas, resultados, re-verificaciones, visualizaciones y documentos maestros— se publica en una carpeta aparte, íntegro y disponible, para que se disponga de la fuente completa de estas ideas; un estudio posterior lo desarrollará en profundidad.

A.2 Naturaleza del trabajo

El estudio fue un ejercicio de indagación que encontró cosas muy interesantes; algunas parecen ciertas y asombrosas, pero el autor, siendo totalmente sincero, no puede corroborarlas, ni lo pretende: esa verificación corresponde a quienes tienen acceso directo a la Sábana. Lo que sí parece con fuerza es que una serie de pruebas y análisis han salido verídicos, y sus resultados —y lo que significarían— son asombrosos; aunque no todos lo han sido. Algunos experimentos parecen bien hechos; otros quizá se acerquen, de algún modo, más a una pareidolia.

A.3 Propósito de esta mención

Todo ese ejercicio —haya producido resultados veraces o no— sirvió para extrapolar conceptos e ideas para el formato. La indagación, los ejercicios y la curiosidad sobre la Sábana Santa permitieron tener esas ideas sobre S-X8 y otras líneas. El formato S-X8, en cambio, **sí ha sido comprobado** (perplexity y benchmarks en este documento). La Sábana solo se menciona para mostrar, en gran parte, de dónde vino la extrapolación de ideas; su veracidad o no es algo que el autor no puede comprobar, pero sin duda da muchísimo que pensar.

A.4 Opinión personal del autor (como tal, explícitamente)

En mi opinión, otros deberían indagar muchísimo más en este tema.

A.5 Transparencia

Se deja el estudio a disposición pública por diversos motivos: intentar ser lo más transparentes posible y que se disponga de la fuente de donde vinieron estas ideas. Se considera lo más correcto, y así se ha hecho.

Apéndice B Direcciones futuras

La familia S-X8 se extiende de forma natural hacia anchos de bit menores. MarlaLabs está desarrollando nuevos formatos y una arquitectura de próxima generación. Estas son direcciones de investigación; en este paper no se reclama ningún resultado sobre ellas.

DOI de Zenodo: 10.5281/zenodo.21922640 · Copyright 2026 Martí Vidal Leandro. Autor: Martí Vidal Leandro (MarlaLabs, Independiente). Datos, código y kernels publicados bajo Apache-2.0. Modelo: Qwen3.5-4B (base Apache-2.0). Hardware: una RTX 5060 Ti de 16 GB. Este documento se maquetoó desde HTML; la fuente LaTeX se incluye junto con él para el envío a arXiv.